



Høyskolen Kristiania

Date: April 23rd - May 7th, 2026

Candidate number: 102

Course code: PG3401 C Programming

Semester: Spring 2026

Task 1**a) Explain what the C programming language can be used for**

The C programming language is a general-purpose, procedural, and structured programming language that was developed in the early 1970s at Bell Laboratories by Dennis Ritchie. It was originally designed for and implemented on the UNIX operating system, replacing the need for assembly language in system-level development (Kernighan & Ritchie, 1988, p. 8). Since its creation, C has grown to become one of the most widely used and influential programming languages in the history of computing.

One of the most important characteristics of C is that it allows a programmer to manipulate bits and bytes directly and gives full control over memory through the use of pointers (Kernighan & Ritchie, 1988, p. 83; Sobell, 2015, p. 11). This makes C particularly well suited for applications where performance and efficiency are critical. At the same time, C is highly portable, meaning that programs written in C can be compiled and run on many different hardware platforms with minimal changes (Kernighan & Ritchie, 1988, p. 8).

C is used across a wide range of application areas. More than 95 percent of the Linux operating system is written in C, and the language is considered ideal for writing compilers and operating systems due to its ability to interact directly with hardware (Sobell, 2015, p. 10, 11). Embedded systems are another major area where C is widely used. Linux, for example, runs in embedded computers such as those found in cellphones and cable boxes (Sobell, 2015, p. 10). C is also the language behind many compilers and language runtimes. The GNU Compiler Collection (GCC), which is the standard C compiler used by Linux and macOS, is itself a product of the C language ecosystem (Sobell, 2015, p. 826).

Beyond its practical applications, C has had a profound influence on the development of other programming languages. C++ was built directly on the foundation of C, and languages such as Objective C also derive from it (Sobell, 2015, p. 11). Languages like Java, C#, and Python also draw heavily from C's syntax and core concepts. Learning C therefore provides a strong foundation for understanding how computers work at a low level and makes picking up other languages considerably easier.

b) Who is Richard Stallman and what is he known for in the field of Information Technology?

Richard Matthew Stallman, commonly known by his initials RMS, is an American programmer and software freedom activist born in 1953. He is widely regarded as one of the most important and controversial figures in the history of computing, particularly in free and open source software. His work has had a lasting impact on how software is developed, distributed, and licensed around the world.

Stallman began his career as a programmer at the MIT Artificial Intelligence Laboratory in 1971, where he became part of a software sharing community in which programs were freely shared and modified (Stallman, n.d.). The situation changed drastically in the early 1980s, when the AI Lab hacker community collapsed and the software industry began shifting toward proprietary models, requiring users to sign nondisclosure agreements even to obtain a copy of a program. Stallman found himself facing what he described as a stark moral choice between joining the proprietary software world or finding another way forward (Stallman, n.d.).

In 1983, Stallman announced the GNU Project, with the goal of creating a complete, free Unix-like operating system. The name GNU stands for "GNU's Not Unix," a recursive acronym reflecting its intention to be compatible with Unix while being entirely free software. In January 1984, he resigned from MIT to begin writing GNU software full

time, ensuring that MIT could not claim ownership of the work or impose proprietary distribution terms (Stallman, n.d.).

In 1985, Stallman created the Free Software Foundation (FSF), a tax-exempt charity dedicated to supporting the development of free software. In the same year, GNU Emacs the first major GNU program was released and made available to the public. Stallman also developed the concept of copyleft, which uses copyright law to ensure that software and all modified versions of it remain free and open (Stallman, n.d.).

Among Stallman's most significant technical contributions are the GNU Compiler Collection (GCC), GNU Emacs, the GNU Debugger (GDB), and the GNU General Public License (GPL). When Linus Torvalds released the Linux kernel in 1991 and made it free software in 1992, it was combined with the GNU system to produce a complete free operating system. Stallman insists this system be called GNU/Linux, to acknowledge the GNU Project's foundational contributions (Stallman, n.d.). This operating system now underpins countless servers, smartphones, and embedded devices around the world.

c) Explain what the sudo command is used for on a Linux-based system

The sudo command is one of the most commonly used commands on Linux-based systems. The name stands for "superuser do," and it allows a regular user to execute commands with elevated privileges, typically as the root user or as another specified user (Sobell, 2015, p. 32). Understanding how and why sudo is used is important for anyone working with Linux systems, whether as a developer, system administrator, or everyday user.

On a Linux system, the root user is the superuser with full administrative access to the entire system. While it is technically possible to log in directly as root and perform all tasks from that account, doing so is generally considered poor security practice. If a user logged in as root makes a mistake, such as accidentally deleting a critical system file, or

if a malicious program is executed, the consequences can be severe and potentially irreversible. The sudo command was designed to address this problem by allowing users to temporarily elevate their privileges only when needed, rather than operating with full root access at all times (Sobell, 2015, p. 32).

When a user runs a command with sudo, they are typically prompted to enter their own password. This confirms that the user intends to perform the action. After authentication, the command is executed with elevated privileges, and the action is logged by the system. This audit trail is an important security feature, as it allows administrators to track who did what and when (Sobell, 2015, p. 33).

The configuration of sudo is managed through a file called `/etc/sudoers`. This file defines which users or groups are allowed to use sudo, and which commands they are permitted to run with elevated privileges. It is possible to grant a user full root access through sudo, or to restrict them to only a specific set of commands. This flexibility makes sudo a powerful tool for managing access control on multi-user systems (Sobell, 2015, p. 32).

Common use cases for sudo include installing or removing software packages using tools such as apt or dnf, editing system configuration files in directories like `/etc`, managing services with systemctl, and performing tasks such as creating or deleting user accounts. In short, any task requiring access to parts of the system restricted to regular users will typically require sudo.

In summary, sudo is a fundamental security tool on Linux systems. It allows users to perform administrative tasks without permanently operating as root, reducing the risk of accidental damage or security breaches. Its combination of access control, password authentication, and command logging makes it an essential part of responsible Linux system administration.

Bibliography

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.).

Prentice Hall.

Sobell, M. G. (2015). A practical guide to Linux commands, editors, and shell

programming (3rd ed.). Prentice Hall.

Stallman, R. (n.d.). The GNU project. GNU Operating System.

<https://www.gnu.org/gnu/thegnuproject.html>

Initialize exam and create folder structure.

- Creating folder structure PG3401/EWA/EXAM_V26_PG3401_EXAM26

```
Eastwill Assistant - EXAM 2026 v1 (id33-DA002243)
```

```
Hi, my name is EWA, I understand that you are having your exam in PG3401.
```

```
How may I help you today?
```

```
(0) Initialize exam and create folder structure  
(2) I want to solve task 2 (simple I/O handling)  
(4) I want to solve task 4 (threaded application)  
(5) I want to solve task 5 (network application)  
(6) I want to solve task 6 (problem solving)  
(9) I want to hear a joke to brighten my day
```

```
Enter selection: 0
```

```
Enter your candidate number: 102
```

```
Info: Creating folder structure..... Done
```

```
Press [Enter] key to continue
```

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26$ ls  
logdata task2_102 task3_102 task4_102 task5_102 task6_102
```

Task 2

File: task2_missing_parameter.png

- Error message if program is started without one parameter or input filename

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_102$ ./task2_program
Error: Missing input file. Usage: ./task2_program <inputfile>
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_102$ █
```

File: task2_scrn1, task2_scrn2, task2_scrn3

- Program running

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_102$ ./task2_program pgexam26_test.txt
Opened input file: pgexam26_test.txt
Index: 0 | Number: 1055
Fibonacci: 1536
Prime Number: 0
Square Number: 1024
Cube Number: 0
Perfect Number: 1024
AbundantNumber: 1024
DeficientNumber: 1025
Odd Number: 1

Index: 1 | Number: 874981
Fibonacci: 1346048
Prime Number: 0
Square Number: 873984
Cube Number: 0
Perfect Number: 896256
AbundantNumber: 896256
DeficientNumber: 896257
Odd Number: 1

Index: 2 | Number: 553
Fibonacci: 512
Prime Number: 0
Square Number: 512
Cube Number: 0
Perfect Number: 512
AbundantNumber: 512
DeficientNumber: 513
Odd Number: 1

Index: 3 | Number: 4243
Fibonacci: 6656
Prime Number: 1
Square Number: 4096
Cube Number: 0
Perfect Number: 4096
AbundantNumber: 4096
DeficientNumber: 4097
Odd Number: 1

Index: 4 | Number: 2442
Fibonacci: 2560
Prime Number: 0
Square Number: 2304
Cube Number: 0
Perfect Number: 5376
AbundantNumber: 5377
DeficientNumber: 5376
Odd Number: 0

Index: 5 | Number: 102
Fibonacci: 0
Prime Number: 0
Square Number: 0
Cube Number: 0
Perfect Number: 0
AbundantNumber: 1
DeficientNumber: 0
Odd Number: 0

Index: 6 | Number: 894
Fibonacci: 768
Prime Number: 0
Square Number: 768
Cube Number: 0
Perfect Number: 1792
AbundantNumber: 1793
DeficientNumber: 1792
Odd Number: 0

Index: 7 | Number: 98732
Fibonacci: 121344
Prime Number: 0
Square Number: 98560
Cube Number: 0
Perfect Number: 172544
AbundantNumber: 172544
DeficientNumber: 172545
Odd Number: 0

Index: 8 | Number: 555
Fibonacci: 512
Prime Number: 0
Square Number: 512
Cube Number: 0
Perfect Number: 768
AbundantNumber: 768
DeficientNumber: 769
Odd Number: 1

Index: 9 | Number: 99273
Fibonacci: 121344
Prime Number: 0
Square Number: 99072
Cube Number: 0
Perfect Number: 132352
AbundantNumber: 132352
DeficientNumber: 132353
Odd Number: 1

Index: 10 | Number: 110283
Fibonacci: 121344
Prime Number: 0
Square Number: 110080
Cube Number: 0
Perfect Number: 146944
AbundantNumber: 146944
DeficientNumber: 146945
Odd Number: 1
```

Successfully created binary output file pgexam26_output.bin

File: task2_scrn4.png, task2_scrn5.png

- EWA After starting menu 2 (scrn4), EWA After verifying pgexam26_output.bin (scrn5)

Eastwill Assistant - EXAM 2026 v1 (id33-DA004243)

Hi, task 2 is a good start on this exam, it is a simple I/O task, to solve it you need to make a makefile to link several files together, the application then needs to read a file, call the new functions on the data and write a new file. If you want I can help to test the result.

I have created a testfile 'pg3401_exam26/task2_102/pgexam26_test.txt' Use this file and the exam text to create your client application.

Ready to test the result? Press 1 to test, 0 to do it later: █

Eastwill Assistant - EXAM 2026 v1 (id33-DA004243)

You have asked me to check your result, let us have a look...

Successfully opened 'pg3401_exam26/task2_102/pgexam26_output.bin'. File is 440 bytes large, let me read it. Successfully read the binary file - check I have checked some of the numbers - check

I have not checked the code quality, that I will leave up to the assessor, but I can read the file and it looks fine. When you think your code quality is good enough this task is done. Good work on this task - keep it up :-)

File: task2_valgrind.png

- Running valgrind to check for memory leaks: *valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task2_program pgexam26_test.txt*

```

==14690==
==14690== HEAP SUMMARY:
==14690==   in use at exit: 0 bytes in 0 blocks
==14690==   total heap usage: 5 allocs, 5 frees, 10,320 bytes allocated
==14690==
==14690== All heap blocks were freed -- no leaks are possible
==14690==
==14690== For counts of detected and suppressed errors, rerun with: -v
==14690== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
odin@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_102$ ls
include  obj          pgexam26_test.txt  task2_cube.c  task2_fib.c  task2_main.c  task2_perf.c  task2_program
makefile pgexam26_output.bin task2_abun.c       task2_def.c   task2_kvad.c  task2_odd.c   task2_prim.c
odin@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_102$ █

```

Task 3

File: task3_scrn1.png

- Running task3_program

```

student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task3_102$ ./task3_program
=====
|          EXAM GRADING SYSTEM - PG3401 Spring 2026          |
=====
| Candidates loaded: 0 |
=====
| 1. Add candidate manually |
| 2. Add candidates from file |
| 3. Assess a task for a candidate |
| 4. Print all candidates |
| 5. Print candidates by grade |
| 6. Print ungraded candidates |
| 7. Print candidate details |
=====
| 9. Exit |
=====
Enter choice: █

```

File: task3_scrn2.png

- Testing function in the program

```

=====
|          EXAM GRADING SYSTEM - PG3401 Spring 2026          |
=====
| Candidates loaded: 1 |
=====
| 1. Add candidate manually |
| 2. Add candidates from file |
| 3. Assess a task for a candidate |
| 4. Print all candidates |
| 5. Print candidates by grade |
| 6. Print ungraded candidates |
| 7. Print candidate details |
=====
| 9. Exit |
=====
Enter choice: 4

=== ALL CANDIDATES ===
Candidate ID: 102
Final Score : 40
Grade       : F
-----

Press <enter> to continue...

```

File: task3_valgrind.png

- Running valgrind to check for memory leaks, after going through all 9 menus in the program: `valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task3_program`

```

=====
| EXAM GRADING SYSTEM - PG3401 Spring 2026 |
=====
| Candidates loaded: 4 |
|-----|
| 1. Add candidate manually |
| 2. Add candidates from file |
| 3. Assess a task for a candidate |
| 4. Print all candidates |
| 5. Print candidates by grade |
| 6. Print ungraded candidates |
| 7. Print candidate details |
|-----|
| 9. Exit |
|-----|
Enter choice: 9

Memory freed.
==14292==
==14292== HEAP SUMMARY:
==14292==    in use at exit: 0 bytes in 0 blocks
==14292==   total heap usage: 8 allocs, 8 frees, 13,096 bytes allocated
==14292==
==14292== All heap blocks were freed -- no leaks are possible
==14292==
==14292== For counts of detected and suppressed errors, rerun with: -v
==14292== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
student@osboxes: ~/PG3401/EWA/EXAM_V26/pg3401_exam26/task3_102$

```

Task 4

File: task4_scrn1.png

- Verifying Part 1 EWA

```

Eastwill Assistant - EXAM 2026 v1 (id33-DA002343)

You have asked me to check your result, let us have a look...
Successfully opened 'pg3401_exam26/task4_102/task4_threads.c' - check
Changes detected in file (hash 0d5d1eb1)
Successfully read the source file - check
Successfully found a makefile in the task4 folder - check
fopen not changed (still hardcoded)
Successfully found mutex use - check
Successfully found explicit init - check
Successfully found 28 comments - check

Part I approved, press [Enter] to get Part II

Press [Enter] key to continue
Copyright Eastwill Security 2026

```

File: task4_scrn2.png

- EWA Assigning part II

```

Eastwill Assistant - EXAM 2026 v1 (id33-DA002343)

Nice work on Part I, I have not checked the quality of the code,
that I will leave up to the assessor - but you have passed my
automated verification and can continue to Part II. Please ensure
it compiles and runs correctly before submitting your exam.
PART II:
The reason why I needed your help to calculate prime numbers is that
a file I have is encrypted by XTEA using one of those prime numbers.
I found the hash of the decrypted file. Could you crack task4_code.bin?
- For each prime your threads find try to decrypt the file
- The first 5 characters are known to be BENGt, check for that result
- If you can crack it, save the file as task4_plain.txt
- Calculate a IP-checksum of the file, write hash to task4_plain.hash
- Once you find a match signal the other threads so both can quit asap
Base your code on the samples I created for you; ipc.c and xtea.c.

Press [Enter] key to continue
Copyright Eastwill Security 2026

```

File: task4_scrn3.png

- Running program, task 1 (prime numbers), task 2 (key found!)

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task4_102$ ./task4 task4_primes.txt
- Loaded task4_code.bin (56 bytes)
```

```
*** KEY FOUND: 1000000009 ***
```

```
Prime numbers found :
```

```
1000000009
1000000033
1000000007
1000000021
```

-

File: task4_scrn4.png

- Valgrind checking for memory leaks: *valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task4 task4_primes.txt*

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task4_102$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task4 task4_primes.txt
==14317== Memcheck, a memory error detector
==14317== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==14317== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==14317== Command: ./task4 task4_primes.txt
==14317==
- Loaded task4_code.bin (56 bytes)
```

```
*** KEY FOUND: 1000000009 ***
```

```
Prime numbers found :
```

```
1000000033
1000000009
1000000021
1000000007
```

```
==14317==
==14317== HEAP SUMMARY:
==14317==      in use at exit: 0 bytes in 0 blocks
==14317==    total heap usage: 24 allocs, 24 frees, 22,150 bytes allocated
==14317==
==14317== All heap blocks were freed -- no leaks are possible
==14317==
==14317== For counts of detected and suppressed errors, rerun with: -v
==14317== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task4_102$
```

Task 5

File: task5_scrn1.png, task5_scrn2.png)

- Running program, running on port 12345 (task5_scrn1.png)
- Connecting EWA to port 12345 (task5_scrn2.png)

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task5_102$ ./task5 -port 12345 -id 102
- [INFO] Starting server id='102' on port 12345
- [INFO] Server listening on 127.0.0.1:12345
- [INFO] Waiting for client connection...
- [INFO] Connection accepted from 127.0.0.1
*** Running SendServerAccept() ***
[INFO] Sent server accept ("127.0.0.1 SMTP 102 06 May 2026, 12:14:31") to client
*** Running HandleClientHello() ***
[INFO] Sent reply ("250 Hello bengt at 127.0.0.1") to client
*** Running HandleMailFrom() ***
[INFO] Sent reply ("250 Got MAIL FROM: bengt@test.com") to client
*** Running HandleRcptTo() ***
[INFO] Sent reply ("250 Got RCPT TO: student@kristiania.no") to client
*** Running HandleData() ***
[INFO] Sent reply ("354 Valid filename: to_33-DA002143.eml") to client
[INFO] Filename accepted: "to_33-DA002143.eml"
[INFO] Setting acCommandStatus to TRAN
*** Running HandleTransfer() ***
[INFO] Awaiting 12 bytes of file data
[INFO] Received 12 bytes, running total: 12
[INFO] Sent reply ("250 Got the data") to client
[INFO] Found end of transfer marker, closing transfer loop
[INFO] File content written to "to_33-DA002143.eml" successfully
*** Running HandleQuit() ***
[INFO] Sent reply ("221 Goodbye! Shutting down the server now.") to client
- [INFO] Client session ended.
- [INFO] Server shutting down. Goodbye.
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task5_102$
```

Eastwill Assistant - EXAM 2026 v1 (id33-DA002143)

Hi, you are still here? Great - let us test your network application :-)
You need to start your server first, I will connect to 127.0.0.1.

Which port number should I connect to?
Port number: 12345

Copyright Eastwill Security 2026

File: task5_scrn3.png, task5_scrn4.png

- EWA Verifying task 5 (task5_scrn3.png)
- Valgrind checking for memory leaks (task5_scrn4.png)

```

Eastwill Assistant - EXAM 2026 v1 (id33-DA002143)

Successfully connected to 127.0.0.1, port 12345
Successfully received: EWP0064|250 127.0.0.1 SMTP 102 06 May 2026, 12:1...
Successfully sent: EWP0064|HELO bengt.127.0.0.1
Successfully received: EWP0064|250 Hello bengt at 127.0.0.1
Successfully sent: EWP0064|MAIL FROM: <bengt@test.com>
Successfully received: EWP0064|250 Got MAIL FROM: bengt@test.com
Successfully sent: EWP0064|RCPT TO: <student@kristiania.no>
Successfully received: EWP0064|250 Got RCPT TO: student@kristiania.no
#1 Successfully sent: EWP0064|DATA to 33-DA002143.eml
#1 Successfully received: EWP0064|354 Valid filename: to_33-DA002143.eml
#1 Successfully sent: EWP0064|QUIT
#1 Successfully received: EWP0064|250 Got the data
Successfully sent: EWP0064|QUIT
Successfully received: EWP0064|221 Goodbye! Shutting down the server now.

Press [Enter] key to continue
Copyright Eastwill Security 2026

```

```

student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task5_102$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task5 -port 12345 -id 102
==3434== Memcheck, a memory error detector
==3434== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==3434== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==3434== Command: ./task5 -port 12345 -id 102
==3434==
--3434-- Starting server id=102 on port 12345
--3434-- [INFO] Server listening on 127.0.0.1:12345
--3434-- [INFO] Waiting for client connection...
--3434-- [INFO] Connection accepted from 127.0.0.1
--3434-- *** Running SendServerAccept() ***
--3434-- [INFO] Sent server accept ("127.0.0.1 SMTP 102 06 May 2026, 12:15:59") to client
--3434-- *** Running HandleClientHello() ***
--3434-- [INFO] Sent reply ("250 Hello bengt at 127.0.0.1") to client
--3434-- *** Running HandleMailFrom() ***
--3434-- [INFO] Sent reply ("250 Got MAIL FROM: bengt@test.com") to client
--3434-- *** Running HandleRcptTo() ***
--3434-- [INFO] Sent reply ("250 Got RCPT TO: student@kristiania.no") to client
--3434-- [INFO] Sent reply ("354 Valid filename: to_33-DA002143.eml") to client
--3434-- [INFO] Filename accepted: "to_33-DA002143.eml"
--3434-- [INFO] Setting acCommandStatus to TRAN
--3434-- *** Running HandleTransfer() ***
--3434-- [INFO] Awaiting 12 bytes of file data
--3434-- [INFO] Received 12 bytes, running total: 12
--3434-- [INFO] Sent reply ("250 Got the data") to client
--3434-- [INFO] Found end of transfer marker, closing transfer loop
--3434-- [INFO] File content written to "to_33-DA002143.eml" successfully
--3434-- *** Running HandleQuit() ***
--3434-- [INFO] Sent reply ("221 Goodbye! Shutting down the server now.") to client
--3434-- [INFO] Client session ended.
--3434-- [INFO] Server shutting down. Goodbye.
==3434==
==3434== HEAP SUMMARY:
==3434==    in use at exit: 0 bytes in 0 blocks
==3434==   total heap usage: 14 allocs, 14 frees, 12,712 bytes allocated
==3434==
==3434== All heap blocks were freed -- no leaks are possible
==3434==
==3434== Far counts of detected and suppressed errors: rerun with: -v
==3434== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task5_102$

```

Task 6

File: task6_scrn1.png

- EWA Task 6 description

```

Eastwill Assistant - EXAM 2026 v1 (id33-DA006043)

Now we have come to the last task, hopefully you mastered the rest. Is is
a problemsolving task, and you need to develop a network client that I
can talk to, you need to add error handling to your protocol handling.
This is a difficult task, I suggest that you attempt this task last ;)

I have started a network server on 127.0.0.1, port 42051, connect to it
and I will send you a file - I will be using the custom TCP-over-TCP
protocol, expect to need to handle malformed packages, out-of-sequence
packages, and other error situations you need to handle. Note that my
behaviour will be random and different each time you test, so your
code needs to be written DYNAMIC, a hardcoded attempt will fail.
Write your client code, I will continue once I have sent the file.

Tip: You can restart me as many times as you need just end me with Ctrl-C.

Copyright Eastwill Security 2026

```

File: task6_scrn2.png, task6_scrn3.png, task6_scrn4.png

- Running program, valgrind checks memory leaks.
- EWA Verifying task 6, butterfly.bmp successfully received

```

student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task6_102$ ./task6 -server 127.0.0.1 -port 42051
*** PG3401 2026 - Task 6 - TCP Client ***
Connecting to 127.0.0.1:42051
[INFO] Connected to 127.0.0.1:42051
[PKT] Seq=0 Flags=0x00 Size=19402
[CR] Got: 0x0BFE Calc: 0x00C1
[WARN] Checksum mismatch for seq=0
[NACK] Requesting resend for seq=0
[PKT] Seq=0 Flags=0x00 Size=19402
[CR] Got: 0x0BFE Calc: 0x00C1
[WARN] Checksum mismatch for seq=0
[NACK] Requesting resend for seq=0
[PKT] Seq=0 Flags=0x00 Size=19402
[CR] Got: 0x0BFE Calc: 0x00C1
[WARN] Checksum mismatch for seq=0
[WARN] Max NACK retries reached for seq=0, accepting anyway
[STORE] Seq=0 Size=19402 Total=1
[ACK] Acknowledging seq=0
[PKT] Seq=38653 Flags=0x00 Size=19285
[CR] Got: 0x5F61 Calc: 0x73A3
[WARN] Checksum mismatch for seq=38653
[NACK] Requesting resend for seq=38653
[PKT] Seq=38653 Flags=0x00 Size=19285
[CR] Got: 0x5F61 Calc: 0x2CDC
[WARN] Checksum mismatch for seq=38653
[NACK] Requesting resend for seq=38653
[PKT] Seq=38653 Flags=0x00 Size=19285
[CR] Got: 0x5F61 Calc: 0x73A3
[WARN] Checksum mismatch for seq=38653
[WARN] Max NACK retries reached for seq=38653, accepting anyway
[STORE] Seq=38653 Size=19285 Total=2
[ACK] Acknowledging seq=38653
[PKT] Seq=19402 Flags=0x00 Size=19251
[CR] Got: 0x538A Calc: 0x190B
[WARN] Checksum mismatch for seq=19402
[NACK] Requesting resend for seq=19402
[PKT] Seq=19402 Flags=0x00 Size=19251
[CR] Got: 0x538A Calc: 0x62B0
[WARN] Checksum mismatch for seq=19402
[NACK] Requesting resend for seq=19402
[PKT] Seq=19402 Flags=0x00 Size=19251
[CR] Got: 0x538A Calc: 0x190B
[WARN] Checksum mismatch for seq=19402
[WARN] Max NACK retries reached for seq=19402, accepting anyway
[STORE] Seq=19402 Size=19251 Total=3
[ACK] Acknowledging seq=19402
[PKT] Seq=57938 Flags=0x01 Size=18916
[CR] Got: 0xD221 Calc: 0x554A
[WARN] Checksum mismatch for seq=57938

```

```

Eastwill Assistant - EXAM 2026 v1 (id33-DA006043)

Successfully sent packet 0 19402
Successfully received ACK packet for : 0 19402et (expected 0x10)
Successfully sent packet 38653 19285
Successfully received ACK packet for : 38653 19285error was intentional)
Successfully sent packet 19402 19251
Successfully received ACK packet for : 19402 19251error was intentional)
Successfully sent packet 57938 18916
Successfully received ACK packet for : 57938 18916expected 0x10)ntional)

If you received the file correctly it should be a bitmap image
with a butterfly - make sure to include it in your delivery.

Press [Enter] key to continue
Copyright Eastwill Security 2026

```

```

[NACK] Requesting resend for seq=57938
[PKT] Seq=57938 Flags=0x01 Size=18916
[CR] Got: 0xD221 Calc: 0x9AF0
[WARN] Checksum mismatch for seq=57938
[WARN] Max NACK retries reached for seq=57938, accepting anyway
[STORE] Seq=57938 Size=18916 Total=4
[ACK] Acknowledging seq=57938
[INFO] FIN received, transfer complete
[INFO] Packets=4 Bytes=76854
[INFO] Saved 76854 bytes to task6_received.bmp
*** Program finished. Goodbye! ***
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task6_102$

```

- File: task6_scrn5.png, task6_scrn6.png, task6_scrn7.png, task6_scrn8.png
- Running test nr2, with valgrind for checks memory leaks this time
 - No leaks found, EWA verified, .bmp saved

```
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task6_102$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./task6 -server 127.0.0.1 -port 42014
```

```
==16993== Memcheck, a memory error detector
==16993== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==16993== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==16993== Command: ./task6 -server 127.0.0.1 -port 42014
==16993==
*** PG3401 2026 - Task 6 - TCP Client ***
Connecting to 127.0.0.1:42014
[INFO] Connected to 127.0.0.1:42014
[PKT] Seq=0 Flags=0x00 Size=19242
[CR] Got: 0x0BAE Calc: 0x0B6A
[WARN] Checksum mismatch for seq=0
[NACK] Requesting resend for seq=0
[PKT] Seq=0 Flags=0x00 Size=19242
[CR] Got: 0x0BAE Calc: 0x0B01
[WARN] Checksum mismatch for seq=0
[NACK] Requesting resend for seq=0
[PKT] Seq=0 Flags=0x00 Size=19242
[CR] Got: 0x0BAE Calc: 0xE4C3
[WARN] Checksum mismatch for seq=0
[WARN] Max NACK retries reached for seq=0, accepting anyway
[STORE] Seq=0 Size=19242 Total=1
[ACK] Acknowledging seq=0
[PKT] Seq=19242 Flags=0x00 Size=19260
[CR] Got: 0x8CE6 Calc: 0x5469
[WARN] Checksum mismatch for seq=19242
[NACK] Requesting resend for seq=19242
[PKT] Seq=19242 Flags=0x00 Size=19260
[CR] Got: 0x8CE6 Calc: 0xF6A7
[WARN] Checksum mismatch for seq=19242
[NACK] Requesting resend for seq=19242
[PKT] Seq=19242 Flags=0x00 Size=19260
[CR] Got: 0x8CE6 Calc: 0x5469
[WARN] Checksum mismatch for seq=19242
[WARN] Max NACK retries reached for seq=19242, accepting anyway
[STORE] Seq=19242 Size=19260 Total=2
[ACK] Acknowledging seq=19242
[PKT] Seq=57763 Flags=0x00 Size=19091
[CR] Got: 0xA5A1 Calc: 0xF2A3
[WARN] Checksum mismatch for seq=57763
[NACK] Requesting resend for seq=57763
[PKT] Seq=57763 Flags=0x00 Size=19091
[CR] Got: 0xA5A1 Calc: 0xF2A3
[WARN] Checksum mismatch for seq=57763
[NACK] Requesting resend for seq=57763
[PKT] Seq=57763 Flags=0x00 Size=19091
[CR] Got: 0xA5A1 Calc: 0xF2A3
[WARN] Checksum mismatch for seq=57763
[WARN] Max NACK retries reached for seq=57763, accepting anyway
[STORE] Seq=38502 Size=19261 Total=4
[ACK] Acknowledging seq=38502
[INFO] FIN received, transfer complete
[INFO] Packets=4 Bytes=76854
[INFO] Saved 76854 bytes to task6_received.bmp
*** Program finished. Goodbye! ***
==16993==
==16993== HEAP SUMMARY:
==16993==    in use at exit: 0 bytes in 0 blocks
==16993==    total heap usage: 9 allocs, 9 frees, 1,262,174 bytes allocated
==16993==
==16993== All heap blocks were freed -- no leaks are possible
==16993==
==16993== For counts of detected and suppressed errors, rerun with: -v
==16993== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
student@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task6_102$
```

Eastwill Assistant - EXAM 2026 v1 (id33-DA007043)

Successfully sent packet 0 19242
 Successfully received ACK packet for : 0 19242t (error was intentional)
 Successfully sent packet 19242 19260
 Successfully received ACK packet for : 19242 19260error was intentional)
 Successfully sent packet 57763 19091
 Successfully received ACK packet for : 57763 19091expected 0x10)
 Successfully sent packet 38502 19261
 Successfully received ACK packet for : 38502 19261expected 0x10)

If you received the file correctly it should be a bitmap image with a butterfly - make sure to include it in your delivery.

Press [Enter] key to continue

Copyright Eastwill Security 2026

